INNOVATION 2026

Equinox
POWERED BY PURPOSE

SOFTWARE HACKATHON

POWERED BY WILLIAMS-SONOMA

AI-THON

Building the future of retail
with AI & innovation

POTTERY BARN pottery barn kids POTTERY BARN | teen west elm GreenRow

WILLIAMS SONOMA WILLIAMS SONOMA HOME MARK & GRAHAM REJUVENATION

Instructions:

1. Problem Statement Selection

Participants may choose one of the provided problem statements or define their own problem statement. If choosing your own, it must be relevant to the retail industry, including (but not limited to) E-commerce, Supply Chain, Data & Analytics, Infrastructure Security

2. Solution Approach

Each provided problem statement includes a suggested solution/output. Participants are encouraged to go beyond these suggestions and propose unique, creative, and well-justified solutions.

3 Use of AI

The use of AI tools and technologies is highly encouraged across all phases, including: Problem understanding, Solution design, Development (final round) . Participants should be prepared to explain how AI influenced their approach and decisions.

4. Judging Criteria

Submissions will be evaluated based on multiple criteria with different weightages. Participants are strongly advised to review the judging criteria carefully and align their approach accordingly.

5. Submission Requirements

All submissions must be made using the official template provided. Additional material can be added as required.

6. Presentation Format

Each team will have a total of 15 minutes, divided as follows: 10 minutes – Presentation, 5 minutes – Q&A with judges

Judging Criteria

Criteria	Explanation	Weightage
Problem Solution Fit	Ability to understand/clearly state the problem statement. Does the solution address the problem effectively?	15%
AI assisted problem solving/thinking	How effective AI was used to break down the problem statement, thought process used to iterate intelligently with AI, how AI influenced the final design of the solution.	10%
Innovation & Creativity	Is the solution provided unique in nature? How different it is from the current solutions available in the industry	10%
AI Engineering/tooling	How effectively AI tools were used to build the solution. Prompt design/engineering quality, Tool selection (LLMs, agents, MCP, A2A, etc.), Handling hallucinations / errors	20%
Technical implementation & prototype quality	Quality of code generate by AI , architecture , tech stack used, data handling, API's /Frameworks used, Security considerations. How effective is the final prototype	20%
Business impact & Practicality	Real-world usefulness, scalability, and potential adoption	15%
User experience & demo effectiveness	App design, ease of use, navigation, and visual appeal	10%

Problem Statements Categories

Ecommerce

Supply Chain

Infra

Cyber Security

Ecommerce: Smart Mobile App Cart

Improve the Williams Sonoma mobile app cart experience by making it more intelligent, contextual, and helpful for users. The cart should proactively suggest **real time bundling offers**, **complementary products**, and **relevant recommendations** based on items already added to the cart, as well as products saved for later. For example, adding cookware could trigger suggestions for matching utensils or accessories, while saved items can be surfaced at the right moment to encourage conversion. These recommendations should feel personalized and timely, helping customers complete their purchase with confidence while also increasing overall cart value and conversion rates.

Challenges

- Identifying relevant complementary products without overwhelming the user
- Maintaining a clean and intuitive mobile UI while adding intelligent suggestions
- Avoiding performance impact on the cart page
- Handling product availability and inventory scenarios
- Managing state updates when cart items change dynamically

Possible solution (You can come up with your own).

- A mobile cart experience with
  - Smart product recommendations
  - Bundle suggestions (e.g., cookware + accessories)
  - Save-for-later suggestions for unavailable items
- Clean UI design for cart intelligence features
- Basic architecture explaining how recommendations would work
- Demo showing cart updates dynamically with suggestions

Note: Preference is to develop Mobile app, however, web app is also ok as a prototype

Ecommerce: Smart Registry/Wishlist

Enhance the Williams Sonoma registry **experience for weddings, housewarmings, and other special events by introducing an AI-powered smart planner** that learns from similar occasions and adding collaboration features that make it easier for hosts, families, and friends to plan together.

Challenges

- Managing shared registry collaboration
- Handling budget constraints
- Suggesting popular or trending registry items based on occasion.
- Providing a clean registry UI on mobile
- Managing gift tracking

Possible solution (You can come up with your own).

Mobile registry features including

- Any more innovations for Smart Registry would be preferable.
- Budget tracking for registry items
- Group gifting support
- Suggested popular registry items
- Registry dashboard showing progress

Note: Preference is to develop Mobile app, however, web app is also ok as a prototype

Ecommerce :Shopping That Thinks

Imagine a customer who just moved into a new apartment. She wants her living room to feel warm, modern, and personal but she cannot describe it in a search bar. She types "sofa" on the **Williams Sonoma** website, gets 312 results sorted by popularity, scrolls for eight minutes, and leaves without buying anything. **WSI operates 7 brands** Pottery Barn, West Elm, Williams Sonoma, Rejuvenation, and more each with deep catalogs across furniture, lighting, and décor. The right sofa, rug, lamp, and cushions all exist across these brands. But today, nobody is connecting those dots for the customer. She is not searching for a product. She is searching for a home.

Challenges

- Understanding customer intent beyond keywords
- Connecting products across 7 separate brands into a coherent experience
- Personalizing suggestions without knowing explicit preferences
- Making high-consideration purchases feel confident and easy
- Avoiding irrelevant or overwhelming recommendations

Possible Solution (You can come up with your own).

- A room photo upload feature — AI suggests products matching the existing aesthetic
- A cross-brand recommendation engine surfacing a coherent bundle, not individual products
- A browsing behaviour model that infers design style and translates it into suggestions
- A conversational shopping assistant returning a curated shortlist from a few questions
- Visual similarity search show more products that look and feel like this one

Additional Material, see next slide

Supporting Data: Shopping That Thinks

Example Scenarios

Scenario 1 Style Mismatch

Customer browses sofas for 10 mins and leaves. No style context captured, generic results shown.

Scenario 2 Cross-Brand Gap

Customer buys a Pottery Barn sofa but never sees the West Elm rug that pairs perfectly with it.

Scenario 3 Visual Intent

Customer uploads a room photo but gets keyword results instead of aesthetic-matched products.

Scenario 4 Save & Forget

Customer saves 12 items across 3 brands. No AI connects them into a coherent room bundle.

Dataset Description

Participants may use publicly available product datasets or generate synthetic data.

1. Product Catalog

File products.csv

product_id Unique ID
brand WSI brand name
category Product category
style_tags Modern / Coastal etc
price Price in USD
description Product description

2. Browsing Sessions

File sessions.csv

session_id Unique session
customer_id Customer
product_id Viewed product
dwell_seconds Time spent
action view/save/cart
timestamp Event time

3. Purchase History

File purchases.csv

customer_id Customer
product_id Purchased item
order_date Date
brand WSI brand

4. Sample Data

File

```
product_id,brand,category,style_tags,p
rice
PB-1001,Pottery Barn,Sofa,warm
modern,1899
WE-2042,West Elm,Rug,mid-century,349
WS-3012,Williams
Sonoma,Cookware,classic,299

session_id,customer_id,product_id,dwel
l_seconds,action
S001,C101,PB-1001,142,view
S001,C101,WE-2042,67,save
S001,C101,WE-2050,12,view
```

Supply chain: Smart Demand Forecasting for Retail Stores

Retail stores often struggle to decide how much stock they should keep because **customer demand keeps changing**. Sometimes products **run out of stock when customers want them** and sometimes stores end up with **too much unsold inventory**. This happens because demand is affected by many factors like **past sales, seasons, promotions, weather, and festivals**. In this challenge, you need to build a **smart system that predicts how many products will be sold in the next few days** and **suggests how much inventory should be maintained**. The goal is to help stores **avoid stockouts, reduce excess inventory, and make better business decisions**

Challenges

- Predicting demand when it changes due to **season, weather, or events**
- Handling **multiple products across multiple stores**
- Using **multiple data sources together**
- Managing cases where **actual demand is hidden (stockouts)**
- Keeping the solution **simple and fast (56 hours)**
- Converting predictions into **useful inventory decisions**
- Predict past sales data and understand patterns

Possible Solution (You can come up with your own)

- Build a system that
  - Predicts demand for the next 7 days
  - Recommends inventory levels
  - Uses data like
    - Past sales
    - Promotions
    - Weather
    - Events/festivals
- Create a simple dashboard that shows
  - Forecast vs past sales
  - Products at risk of stockout
  - Recommended inventory levels
- Add real-time adjustment for sudden demand changes(Bonus)

Supply Chain: The Supply Chain Sees the Future

It is mid-October. WSI's peak revenue window, Thanksgiving, Black Friday, Christmas is six weeks away. A buyer places a replenishment order for 6,000 units of a best-selling Pottery Barn throw blanket with a 10-week lead time from a factory in India. What neither she nor the model knows three days ago, a home décor creator with 2.4 million followers posted a reel featuring that exact blanket. Demand is now tracking toward 18,000 units. The order cannot be changed. Meanwhile 4,200 units of a decorative pillow sit unsold in a separate warehouse over-ordered on a trend that peaked in February. Overstock and stockout happening simultaneously in the same company because the system cannot see both problems at once.

Challenges

- Demand forecasting is backward-looking blind to social and trend signals
- The window to act on a signal is shorter than lead times in home furnishings
- Overstock and stockout occur simultaneously with no unified view
- External signals (social media, search, events) are unstructured and hard to fuse
- Buyers need plain-language guidance, not just raw model output

Possible Solution (You can come up with your own).

- A demand signal fusion model combining internal sales data with external signals (social, search, events)
- An early warning system flagging at-risk SKUs weeks before stockout or overstock becomes inevitable
- A plain-language weekly brief for buyers which SKUs to act on and why
- An anomaly detector that flags when a product's demand velocity suddenly changes
- A simulation showing what would have happened if the signal had been caught earlier

Additional Material, see next slide

Supporting Material: The Supply Chain Sees the Future

Example Scenarios

Scenario 1 Viral Spike

Influencer post drives 3x demand overnight. Lead time = 10 weeks. Stockout inevitable.

Scenario 2 Silent Overstock

4,200 units of a pillow unsold. Trend peaked 6 months ago. No system flagged it.

Scenario 3 Seasonal Blindspot

Forecast misses a festival-driven demand spike in a specific region.

Scenario 4 Correlated Signals

Rising housing permits + social trend together predict blanket demand — neither alone is enough.

Dataset Description

Participants may use publicly available retail/sales datasets or generate synthetic data.

1. Historical Sales

File sales.csv

sku_id Product SKU
date Sale date
units_sold Units sold
store_id Store/channel
category Product category
price Sale price

2. External Signals

File signals.csv

date Signal date
sku_id Related SKU
signal_type social/search/event
volume Signal intensity
source Platform/source

3. Inventory Positions

File inventory.csv

sku_id Product SKU
warehouse_id Location
stock_on_hand Current units
on_order Units on order
lead_time_days Days to restock

4. Sample Data

File

```
sku_id,date,units_sold,category
PB-BLANKET-42,2026-01-01,120,Bedding
PB-BLANKET-42,2026-01-08,118,Bedding
PB-BLANKET-42,2026-03-14,412,Bedding

date,sku_id,signal_type,volume
2026-03-12,PB-BLANKET-
42,social,2400000
2026-03-13,PB-BLANKET-42,search,340
```

Supply Chain: Intelligent Warehouse Slotting Optimization

Warehouses often store products in random places, which makes it harder and slower for workers to pick items. This leads to **longer travel time lower efficiency**, and sometimes **unsafe storage of heavy items.** In this challenge, you need to build a **smart system that decides where each product should be placed in a warehouse**. The system should help reduce travel distance, improve picking speed, and organize products better. The goal is to **minimize picker movement**, **place fast-moving items closer to dispatch**, and **keep related products near each other** so that orders can be fulfilled faster and more efficiently.

Challenges

- Deciding the best place for each product in a limited space
- Placing **fast-moving products near dispatch**
- Ensuring **heavy products are stored safely**
- Identifying **products frequently bought together**
- Balancing multiple factors at the same time
- Simulating and calculating **picker travel distance**
- Keeping the solution simple and fast (56 hours)
- Converting placement logic into a clear visual output

Possible Solution (You can come up with your own)

- Build a system that
  - Assigns the **best slot for each product**
  - Minimizes **total travel distance**
- Follows rules like
  - Fast-moving items → near dispatch
  - Heavy items → safe/lower positions
  - Related items → close to each other
- Use data like
  - Product demand / sales frequency / Product weight / Warehouse slot locations / Order history
- Create a simple dashboard that shows
  - Current vs optimized layout / Travel distance before vs after / Products placed near dispatch / Frequently bought together items

Supply chain: Returns Intelligence: Every Return Is a Signal

A customer orders a West Elm dining table. She assembles it. Then she returns it, selecting **"does not match description"** from the dropdown. The real reason the tabletop looked warm in product photos but pulls cool and grey in her dining room. WSI processes the return outbound shipping, return shipping, reprocessing approximately **$180 in costs on a $640 sale.** That same table has been returned 34 times this quarter. 28 picked the same reason. 11 reviews mention "color looks different in real life." All this data exists in separate systems, in different formats, completely unconnected.

Challenges

- Return dropdown reasons are imprecise real signal is buried in free text
- Returns, reviews, and customer service contacts sit in separate, unconnected systems
- Product teams have no feedback loop until damage is already done
- The gap between on-screen appearance and in-home reality is a structural problem
- High return rates erode margin silently without triggering any visible alert

Possible Solution (You can come up with your own).

- An NLP system reading across return notes, reviews, and contacts to surface real SKU-level issues
- A return-risk scorer given a listing, predict the likelihood of return before the item ships
- A tool connecting return patterns to specific listing attributes causing customer confusion
- A pre-purchase visualization feature reducing the 'looks different in real life' return reason
- A merchandising dashboard prioritizing which listing fixes have the highest impact

Additional Material, see next slide

Supporting Material: Returns Intelligence Every Return Is a Signal

Example Scenarios

Scenario 1 Colour Mismatch

Table looks warm in photography but renders cool in real lighting. 28 returns, same dropdown reason.

Scenario 2 Size Confusion

Sofa dimensions listed correctly but customers can't visualise scale. 19 returns citing 'wrong size'.

Scenario 3 Linked Signal

34 returns + 11 reviews + 2 CS chats all pointing to same listing issue. Nobody connected them.

Scenario 4 Category Pattern

A product category has 3x return rate vs. average. Root cause photography style, not product quality.

Dataset Description

Participants may generate synthetic data or use publicly available review/return datasets.

1. Return Records

File returns.csv

return_id Unique return
order_id Original order
sku_id Product SKU
return_reason Dropdown reason
return_note Free text (optional)
return_date Date returned

2. Product Reviews

File reviews.csv

review_id Unique review
sku_id Product SKU
rating 15 stars
review_text Customer review
review_date Date posted

3. CS Contacts

File cs_contacts.csv

contact_id Unique contact
sku_id Product SKU
contact_reason Issue category
transcript Chat/call notes
resolution How resolved

4. Sample Data

File

```
return_id,sku_id,return_reason,return_
note
R001,WE-TABLE-88,does_not_match,colour
looked different
R002,WE-TABLE-88,does_not_match,warmer
in the photo
R003,WE-TABLE-88,no_longer_needed,

sku_id,rating,review_text
WE-TABLE-88,2,colour is not what I
expected
WE-TABLE-88,3,looks grey not warm
brown
```

Supply chain: Data Quality Clean Data In, Everything Works

A vendor submits a spreadsheet of 200 new SKUs. The item setup team has 48 hours. Hidden inside a side table with a height of **4 centimetres**. Wine glasses weighing **45 kilograms**. A 'compact accent mirror' with dimensions of 220cm × 180cm larger than a king-size bed. A product description copy-pasted from a completely different SKU. Seven items that may already exist in the catalog under different names. None of this is caught before go-live. Three months later wrong warehouse bins, split search results, freight anomalies, and a customer service escalation. The errors were never malicious. They were invisible.

Challenges

- Errors come in many forms structural, semantic, relational no single check catches all
- Volume makes manual review impossible hundreds of vendors, thousands of SKUs per season
- Duplicate SKUs with different names are invisible to simple validation rules
- Errors at intake compound silently as they move into warehousing, freight, and the catalog
- Vendors need explanations they can act on, not just rejection flags

Possible Solution (You can come up with your own).

- An AI intake validator scoring incoming records and flagging what looks wrong or implausible
- A duplicate detection system using text embeddings to identify SKUs already in the catalog
- An LLM-assisted feedback tool explaining errors in plain language with suggested corrections
- A vendor reliability model predicting how much manual review a new submission actually needs
- A visual anomaly detector for product images flagging photos that don't match the declared category

Additional Material, see next slide

Supporting Data: Data Quality Clean Data In, Everything Works

Example Scenarios

Scenario 1 Impossible Dimension

Side table height = 4cm. Mirror dimensions = 220×180cm. Both pass format check, both are wrong.

Scenario 2 Copy-Paste Error

Product description for a ceramic vase is identical to a SKU submitted 3 months ago by same vendor.

Scenario 3 Duplicate SKU

New SKU 'Coastal Throw Pillow XL' is the same product as 'Beach Cushion Large' submitted last season.

Scenario 4 Unit Confusion

Wine glass weight = 45kg. Vendor entered grams (450g) in the kilograms field.

Dataset Description

Participants may simulate vendor submission datasets or use the structure below.

1. Vendor Submissions

File vendor_items.csv

sku_id Vendor SKU code
vendor_id Vendor ID
category Product category
each_l_cm Unit length (cm)
each_w_cm Unit width (cm)
each_h_cm Unit height (cm)
each_wt_kg Unit weight (kg)
description Product description

2. Existing Catalog

File catalog.csv

wsi_sku_id WSI catalog ID
product_name Product name
category Category
description Current description
l_cm Length (cm)
w_cm Width (cm)
h_cm Height (cm)

3. Vendor History

File vendor_history.csv

vendor_id Vendor ID
submission_date Submission date
total_skus SKUs submitted
error_count Errors found
error_rate_pct Error rate %

4. Sample Data

File

```
sku_id,vendor_id,category,each_l_cm,ea
ch_w_cm,each_h_cm,each_wt_kg
V001-ST,V001,Side Table,45,40,4,8.2
V001-MR,V001,Mirror,220,180,3,2.1
V002-WG,V002,Glassware,9,9,22,45.0
```

```
vendor_id,total_skus,error_count,error
_rate_pct
V001,200,34,17.0
V002,85,52,61.2
```

Infrastructure: Zero Trust Access Policy Validator

Retail enterprises operate with complex access control structures across, Stores (POS systems, inventory tools), Warehouses (supply chain systems), Corporate systems (HR, finance, analytics), eCommerce platforms To manage access, organizations rely on Users → Groups → Roles → Permissions → Resources. Over time, this leads to A) Over-Privileged Access - Users have access they don't need violating least privilege principal B) User Group Sprawl - Too many groups created over time, Groups with overlapping permissions which is difficult to maintain and audit. C) Redundant & Unused Groups - Groups no longer in use or duplicate groups with same access and lastly all these add to D) Operational overheads including high cost of maintenance. Build a system that analyses users, roles, permissions, and user groups to identify over-privileged access, redundant groups, and recommend optimized, least-privilege access policies. The system should improve both Security posture Operational efficiency (reduced group complexity)

Challenges

- Over privileged across Users have access they don't need
- Complex hierarchy users → groups → roles → permissions
- User group sprawl and overlapping permissions
- Identifying unused or redundant access
- Balancing security with usability
- Lack of clear visibility into access patterns

Possible solution (You can come up with your own).

A system/application that can

- Provide list of over-privileged users
- Identify Redundant / duplicate groups
- Provides group consolidation recommendations
- Optimized access model (reduced groups)
- Dashboard showing access risk and improvements

Additional Material, see next slide

Supporting Data: Zero Trust Access Policy Validator

Example Scenarios

Scenario 1 Duplicate Groups
Group A → Inventory Read
Group B → Inventory Read
Same permissions → should be merged

Scenario 2 Overlapping Groups
Group A → Inventory + Orders
Group B → Inventory + Orders + Pricing
High overlap → optimize

Scenario 3 Unused Groups
•Group exists but no users assigned should be removed

Scenario 4 Over-Privileged User
•Store user has HR system access unnecessary

Dataset Description

Participants may simulate or use structured datasets.

1. User Data

File: users.csv

Column	Description
user_id	Unique user
department	Store / Warehouse / Corporate
role	Job role

2. Group Mapping

File: user_groups.csv

Column	Description
user_id	User
group_id	Group assigned

3. Group Permissions

File: group_permissions.csv

Column	Description
group_id	Group
resource	System
permission	read / write / admin

Sample Data

```
user_id,group_id
emp101,grp_store_basic
emp101,grp_inventory_admin
group_id,resource,permission
grp_store_basic,inventory_db,read
grp_inventory_admin,inventory_db,admin
grp_inventory_admin,pricing_db,write
```

Infrastructure: Real-Time Insider Threat Detection

Retail enterprises operate across **stores, warehouses, and digital platforms**, where thousands of employees access systems daily. While external threats are commonly addressed, **insider threats remain one of the most difficult to detect** because Actions appear legitimate or Access is already authorized or behaviour deviations are subtle. Organizations need intelligent systems that can **analyse activity patterns and detect anomalies in real time**. Build an AI-based system that analyses enterprise user activity logs to detect abnormal behaviour and identify potential insider threats such as Unusual login patterns, Unauthorized access to sensitive systems, Abnormal data transfer activity, Privilege misuse. The system should generate **risk scores, alerts, and actionable insights**.

Challenges

- Differentiating normal vs abnormal behaviour
- Lack of labelled data for threats
- Detecting subtle anomalies (not obvious attacks)
- Handling large volumes of logs
- Minimizing false positives

Possible solution (You can come up with your own).

- Risk score per user/session
- List of suspicious users/activities
- Alerts for anomalous behaviour
- Explanation of why user was flagged
- Dashboard with activity timeline and risk insights

Additional Material, see next slide

Supporting Data: Real-Time Insider Threat Detection

Example Scenarios

Scenario 1 Off-Hours Access
A warehouse employee logs in at **230 AM**, which is outside normal working hours.

Scenario 2 Data Exfiltration
A user downloads **5GB of customer data**, significantly higher than usual behavior.

Scenario 3 Unauthorized Resource Access
A store employee accesses **payroll or HR systems**, which are not part of their role.

Scenario 4 Multiple Location Logins
A user logs in from
- Bangalore
- then Singapore within 10 minutes

Scenario 5 Privilege Misuse
A user suddenly
- performs admin-level actions
- modifies system configurations

Dataset Description

Participants may use publicly available datasets or generate synthetic data

1. User Activity Logs

File user_activity_logs.csv

Column	Type	Description
timestamp	datetime	Time of action
user_id	string	Unique user identifier
role	string	User role
action	string	login / access / download
resource	string	System or database accessed
location	string	City/location
data_volume_mb	float	Data accessed/downloaded
device	string	Device type
status	string	success / failure

Sample Data

```
timestamp,user_id,role,action,resource,location,data_volume_mb,device,status
2026-03-10 0900,emp101,store_user,login,system,Bangalore,0,laptop,success
2026-03-10 0215,emp101,store_user,download,customer_db,Unknown,5000,laptop,success
```

2. User Role Mapping

File user_roles.csv

Column	Description
user_id	User ID
department	Department
expected_resources	Allowed systems

Infrastructure: Log & Metrics Anomaly Detection

Modern retail platforms (stores + eCommerce + backend systems) generate massive volumes of logs and metrics, including application logs (checkout, cart, payments), infrastructure metrics (CPU, memory, disk), service health metrics (latency, error rate), Operations teams face challenges1. Signal vs Noise Problem (Thousands of logs per second, Hard to identify real issues), 2. Unknown Errors (New error types appear without prior patterns), 3. Error Spikes (Sudden increase in known errors goes unnoticed), 4. Lack of Correlation (Logs and metrics analyzed separately, root cause unclear). Build an intelligent system that analyzes logs and infrastructure metrics to detect new/unseen errors, spikes in existing errors, anomalies in system metrics. The system should correlate signals and generate actionable alerts with context.

Challenges

- High volume of logs (noise vs signal)
- Detecting new/unseen error patterns
- Identifying spikes in known errors
- Correlating logs with metrics (CPU, latency, etc.)
- Avoiding alert fatigue

Possible solution (You can come up with your own).

- Detection of new error types
- Alerts for error spikes and metric anomalies
- Correlated insights (e.g., error + CPU spike)
- Severity classification of issues
- Dashboard showing trends and anomalies

Additional Material, see next slide

Supporting Data: Log & Metrics Anomaly Detection

Example Scenarios

Scenario 1 Error Spike
- Checkout failures increase from **1% → 25% within 5 minutes**

Scenario 2 New Error
- New log message appears `PaymentServiceTimeoutException`

Scenario 3 CPU Spike
- CPU jumps to 95% → service degradation

Scenario 4 Latency Surge
- response time increases from 100ms → 800ms

Scenario 5 Combined Failure
- CPU spike + error spike + latency increase
  critical incident

Dataset Description

1. Application Logs
File app_logs.csv

Column	Description
timestamp	time
service	service name
log_level	INFO / ERROR
error_code	error type
message	log message

2. Infrastructure Metrics
File infra_metrics.csv

Column	Description
timestamp	time
cpu_usage	%
memory_usage	%
disk_usage	%

3. Service Metrics
File service_metrics.csv

Column	Description
timestamp	time
service	name
latency_ms	response time
error_rate	% errors

Sample Data

```
timestamp,service,log_level,error_code,message
1001,checkout,INFO,NA,request_success
1005,checkout,ERROR,500,payment_failed
1006,checkout,ERROR,500,payment_failed
timestamp,cpu_usage,memory_usage,disk_usage
1000,40,60,50
1010,95,85,70
```

Infrastructure: : AI-Driven Automation Workflows

Engineering teams spend **60%+ time on repetitive operational tasks**, including responding to alerts, restarting services, resolving incidents, generating reports. **Key Problems 1. Manual Operations (**slow response times), **2. Fragmented Workflows (**multiple tools, no integration), **3. Lack of Intelligence (**no decision-making automation), **4. Delayed Resolution (**issues escalate unnecessarily). Design a system that automates incident handling by detecting issues, triggering alerts, executing resolution workflows, and generating incident reports with minimal manual intervention. Detection → Alert → Decision → Action → Resolution → Reporting

Challenges

- Mapping alerts to correct actions
- Designing reliable workflow logic
- Handling multiple/complex incident scenarios
- Ensuring safe automation (avoid wrong actions)
- Integrating detection → action → resolution

Possible solution (You can come up with your own).

•Automated workflow execution (alert → action)
•Recommended or executed remediation steps
•Incident tracking and status (resolved/pending)
•End-to-end incident summary/report
•Dashboard showing automation efficiency

Additional Material, see next slide

Supporting Data: AI-Driven Automation Workflows

Example Scenarios

Scenario 1 High CPU
- CPU > 90%
  restart service

Scenario 2 Disk Full
- disk usage > 95%
  cleanup logs

Scenario 3 API Failure
- API returns 500 errors
  switch to fallback

Scenario 4 Service Down
- health check fails
  restart + notify

Scenario 5 Repeated Failure
- issue persists after action
  escalate

Dataset Description

1. Alerts
File alerts.csv

Column	Description
timestamp	time
alert_type	type
severity	level
service	affected service

2. Workflow Rules
File workflows.csv

Column	Description
alert_type	alert
action	action
priority	execution order

3. Incident Logs
File incidents.csv

Column	Description
incident_id	id
alert_type	alert
action_taken	action
status	resolved/pending

Sample Data

```
timestamp,alert_type,severity,service
1005,high_cpu,critical,checkout
alert_type,action
high_cpu,restart_service
disk_full,cleanup_logs
```

Wild Card

Pick any problem statement related to retail industry in the following categories:

- Ecommerce
- Supply chain/Logistics
- Infrastructure/Cloud
- Cyber security